

MONJED

AI Rescue Robot Digital Twin

Implementation Report

What was built, how it works, where it lives in the codebase, and how it prepares MONJED for real Arduino/ESP32 hardware.

Document date: 2026-09-06

Product: MONJED multi-hazard early warning and action (Africa)

Feature route: /admin/rescue-robot (admin only)

API prefix: /rescue-robot (separate from Leaflet /simulation)

Contents

1. Executive summary
2. Relationship to the older map simulation
3. Product goal and design principles
4. End-to-end demo workflow
5. Architecture (sim-to-hardware)
6. Robot model, sensors, and arm
7. 3D environment and disaster scenarios
8. Navigation and autonomous mission controller
9. MONJED command-center UI
10. Backend API and telemetry contract
11. Code map (folders and key files)
12. How to run and demonstrate
13. Future hardware integration path
14. What is intentionally out of scope (V1)

1. Executive summary

We built a new interactive MONJED feature: an AI Rescue Robot Digital Twin. It is a real software simulation (state, sensors, missions, battery, obstacles, victims, event log), not a fake point-to-point animation. The twin is designed so simulated hardware drivers can later be replaced by ESP32/Arduino/ROS drivers without rebuilding the MONJED mission UI.

Delivered in this implementation:

- Admin command center at /admin/rescue-robot with 3D viewport and ops panels.
- Robot MONJED-R01: differential-drive base, ultrasonics, temp, gas, IMU, battery, GPS, camera, robotic arm and gripper.
- Scenarios: Flood, Earthquake, Fire - each with different hazards and behaviors.
- Autonomous mission controller: create, dispatch, navigate, avoid, inspect, detect victim, return, complete.
- Demo Mode that runs the full flood response story for judges and partners.
- REST API under /rescue-robot that mirrors missions and telemetry into MONJED.
- Clear hardware abstraction interfaces (Motor, Sensor, Arm, Telemetry, Mission).

2. Relationship to the older map simulation

MONJED already had a Leaflet-based fleet ops tool at /admin/simulation (roads, drones/rovers, help-queue dispatch). That tool remains unchanged.

The new digital twin is intentionally separate:

- Route: /admin/rescue-robot (Admin sidebar: "AI rescue robot").
- API: /rescue-robot/* - not /simulation/*.
- Frontend folders: components/rescueRobot and lib/rescueRobot.
- Purpose: 3D digital twin and sensor/arm mission loop for a future physical robot.

Keeping them separate avoids mixing 2D live-ops GPS dispatch with the hardware-oriented twin, and lets each evolve independently.

3. Product goal and design principles

MONJED's loop is: Sense risk -> understand -> decide -> alert -> deploy help -> track response. The rescue robot extends deploy/track with a robotic responder.

Principles used while building

- Real state machine - idle, navigating, avoiding, inspecting, victim detected, returning, completed, emergency, failed, low battery.
- Sensors drive behavior - close ultrasonics trigger avoidance; fire temp/gas can force emergency retreat; low battery forces return-to-base.
- Stable telemetry JSON - same shape for sim today and hardware later.
- Start small, make it work - engine + 3D + missions first; then arm, victims, scenarios, demo.
- Integrate into MONJED - React admin auth, Tailwind tokens, FastAPI router registration, existing api.js helpers.

4. End-to-end demo workflow

Demo Mode (button in the command center) runs this story automatically:

- FLOOD DETECTED - high risk sector loaded in 3D
- Affected area identified (Sector A / Building 04)
- Mission created (Search and Rescue, HIGH priority)
- MONJED-R01 dispatched from safe-zone base
- Robot navigates with A* path around flooded / blocked cells
- Obstacle detected -> route recalculated
- Sensors continuously publish temp, gas, distances, battery, pose
- Possible victim detected near Building 04 -> alert + coordinates to MONJED
- Inspection completed at target
- Robot returns to base
- Mission completed - event log shows the full timeline

Operators can also manually Create Mission, Dispatch, Pause, Resume, Return to Base, or Stop. Scenario selector switches Flood / Earthquake / Fire.

5. Architecture (sim-to-hardware)

Communication flow implemented:

```
MONJED Command UI
|
Mission Controller (engine.js)
|
SimulatedMotors / SimulatedSensors / SimulatedArm
|
3D World + Physics tick (requestAnimationFrame)
|
Telemetry JSON mirror --> POST /rescue-robot/telemetry
|
MONJED mission/event store (backend)
```

Abstraction modules (frontend/src/lib/rescueRobot/interfaces.js):

- MotorInterface - differential-drive velocity commands and pose.
- SensorInterface - ultrasonic, temperature, gas, IMU, battery, GPS, camera.
- ArmInterface - MOVE_ARM, OPEN/CLOSE_GRIPPER, PICK/DROP_OBJECT, RESET_ARM.
- TelemetryInterface / MissionInterface - publish and lifecycle contracts.

Today: Simulated* classes implement these. Later: Physical* drivers (serial/MAVLink/ESP32 gateway) implement the same methods. MissionController and UI stay the same.

6. Robot model, sensors, and arm

Robot MONJED-R01

- Wheeled differential-drive base (four visible wheels in 3D).
- Status beacon color changes with mission state.
- Camera mast representation and front ultrasonic pod.
- Battery strip on the chassis (visual and numeric telemetry).

Simulated sensors (live every tick)

- Ultrasonic: front / left / right distances (ray checks vs world obstacles).
- Temperature: ambient + fire-zone boost; critical levels trigger emergency in Fire scenario.
- Gas level: NORMAL / ELEVATED / HIGH / CRITICAL from proximity to fire zones.
- IMU: roll/pitch/yaw and moving flag.
- Battery: drains while operating; critical battery auto return-to-base.
- GPS/pose: XZ metres on the twin ground plane.

Robotic arm

- Joints: base, shoulder, elbow, wrist, gripper.
- Commands supported in SimulatedArm.
- Earthquake scenario: approach movable debris, PICK_OBJECT, relocate aside, replan path.

7. 3D environment and disaster scenarios

The first environment is a lightweight flooded urban/community area (roads, buildings, houses, trees, walls, debris, flood pools, safe zone, victims).

Flood

- Impassable flooded zones; robot must path around them.
- Obstacles force avoidance and recalculation.
- Victim near Building 04 for search-and-rescue demo.

Earthquake

- Damaged buildings and debris field.
- Movable debris can be cleared with the arm.

Fire

- Fire zones raise temperature and gas.
- Above safe thresholds -> EMERGENCY status, retreat to base, event logged to MONJED.

Rendering uses Three.js (vanilla, browser WebGL) with drag-orbit camera. React Three Fiber was attempted but dropped due to install/peer issues; vanilla Three keeps the dependency simple (package: three).

8. Navigation and autonomous mission controller

Navigation

- Grid A* planner (navigation.js) treats floods/walls/debris as blocked cells.
- Path simplified with line-of-sight shortcuts when clear.
- While driving: follow waypoints; if ultrasonic/front collision risk -> AVOIDING, rotate, replan (throttled).
- Near destination: treat goal proximity as arrival so the robot can inspect instead of looping on building walls.

Mission phases

- created -> dispatched/en_route -> inspecting -> returning -> completed
- Also: paused, failed/stopped, emergency, low-battery return.

The controller is rule/state-based (reliable for V1). Structure allows swapping in a richer planner later without rewriting the UI.

9. MONJED command-center UI

Full-page admin console (not AppLayout), matching MONJED ops styling:

- Left: large 3D digital twin viewport.
- Right rail: Hazard/Risk, Robot status, Live telemetry, Create mission, Controls, Victim alert, Event log.
- Working controls: Create Mission, Dispatch, Pause, Resume, Return to Base, Stop, Demo Mode, Scenario switch.
- Victim panel: location, confidence, coordinates, "transmitted to MONJED".
- Theme toggle and link back to Admin; note pointing to legacy /admin/simulation.

10. Backend API and telemetry contract

FastAPI router registered in backend/app/main.py:

```
GET /rescue-robot/health
POST /rescue-robot/missions
GET /rescue-robot/missions
GET /rescue-robot/missions/{id}
POST /rescue-robot/telemetry
GET /rescue-robot/telemetry/latest
GET /rescue-robot/events
```

Example telemetry shape mirrored from the twin:

```
{
  "robotId": "MONJED-R01",
  "status": "INSPECTING",
  "battery": 76,
  "temperature": 34,
  "gasLevel": "NORMAL",
  "position": { "x": 14.2, "y": 8.7 },
  "missionId": "MISSION-001",
  "ultrasonic": { "front": 1.8, "left": 3.2, "right": 0.9 },
  "warnings": ["OBSTACLE_AHEAD"]
}
```

Store is in-memory for the twin mirror (fast, demo-friendly). The real-time tick loop stays in the browser for smooth 3D; the API keeps MONJED informed.

11. Code map (folders and key files)

Frontend

- pages/RescueRobot.jsx - page shell, engine subscribe/tick, API mirror.
- components/rescueRobot/RescueScene.jsx - Three.js scene, robot, world, path.
- components/rescueRobot/CommandCenter.jsx - ops panels and controls.
- lib/rescueRobot/engine.js - mission controller and simulation tick.
- lib/rescueRobot/world.js - Flood/Earthquake/Fire scenario definitions.
- lib/rescueRobot/navigation.js - A* and avoidance helpers.
- lib/rescueRobot/simulatedHardware.js - Simulated motors/sensors/arm.
- lib/rescueRobot/interfaces.js - hardware contracts.
- App.jsx route + Admin.jsx nav item "AI rescue robot".
- lib/api.js - createRescueMissionRemote, mirrorRescueTelemetry, etc.

Backend

- routers/rescue_robot.py - HTTP endpoints.
- schemas/rescue_robot.py - Pydantic models.
- services/rescue_robot_store.py - mission/telemetry/event store.

Dependency

- frontend package.json: three ^0.175.0 (vanilla Three.js).

12. How to run and demonstrate

- Start backend (FastAPI) with existing MONJED setup.
- In frontend/: npm install (ensures three), then npm run dev.
- Sign in as admin -> Admin -> AI rescue robot.
- Click Demo Mode, or manually pick scenario -> Create Mission -> Dispatch Robot.
- Watch 3D path, telemetry, event log, and victim alert; wait for return-to-base completion.

Verified in development: production Vite build succeeds; automated engine tick completes flood demo with obstacle + victim + mission completed.

13. Future hardware integration path

Recommended next steps when buying hardware:

- Keep TelemetryFrame / POST /rescue-robot/telemetry unchanged.
- Implement PhysicalMotors, PhysicalSensors, PhysicalArm talking to ESP32/Arduino.
- Gateway service (serial/Wi-Fi/LTE) publishes the same JSON MONJED already accepts.
- Optional: ROS 2 / Gazebo later for higher-fidelity physics - twin UI/API still applies.
- Bill-of-materials can reuse component SKU ideas (FC, GPS, IMU, LiDAR/ultrasonic, camera, motors, arm servos).

14. What is intentionally out of scope (V1)

- Not a replacement for the Leaflet live help-queue fleet map.
- Not industrial physics or Gazebo-level fidelity.
- Not JWT-locked backend routes (matches current MONJED API pattern; admin gate is frontend).
- Not React Three Fiber (install issues); vanilla Three.js instead.
- Telemetry mirror is best-effort if API is offline - local twin still runs.

Closing

MONJED now has a working robotic-response digital twin that demonstrates the full hazard -> mission -> robot action -> sensor data -> MONJED update -> completion loop, architected for a clean swap from simulated hardware to physical rescue-robot hardware.

Primary entry for demos: /admin/rescue-robot -> Demo Mode.